

🔍 Linear Search Algorithm in Python

Definition:

Linear Search (also called Sequential Search) is the simplest search algorithm that checks every element in a list one by one until the desired element is found or the list ends.

✔ Characteristics:

Feature	Description
Time Complexity	$O(n)$ — checks each element once
Space Complexity	$O(1)$ — no extra space needed
Best Case	$O(1)$ — if the target is at the beginning
Worst Case	$O(n)$ — if the target is at the end or not found
Data Requirement	Works on unsorted or sorted lists

❑ How It Works:

- Start from index 0.
 - Compare each element with the target.
 - If match found, return index or element.
 - If not found after checking all elements, return -1 or "Not found".
-

❑ Python Example:

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i # return the index of the found element
    return -1 # target not found

# Example usage
numbers = [5, 3, 7, 1, 9, 2]
target = 1

result = linear_search(numbers, target)
if result != -1:
    print(f"Element found at index: {result}")
else:
    print("Element not found")

# ❑ Output:
# Element found at index: 3
```

✦ When to Use Linear Search?

- When the dataset is small.
 - When the list is unsorted.
 - When performance is not critical.
-

🔍 What is Binary Search?

Binary Search is an efficient algorithm used to find a specific element in a sorted list. It works by dividing the search space in half again and again, reducing the time it takes to find the item.

✓ Prerequisite

- The list **must be sorted** in ascending order (very important).
 - Binary search is more efficient than linear search, especially for large lists.
-

💡 Idea (Analogy)

Suppose you are looking for a number in a book that contains sorted page numbers. Instead of going page by page, you:

- Open the book in the middle.
- Check if it's the page you're looking for.
- If not, decide whether to go left (lower numbers) or right (higher numbers).
- Repeat until you find the page.

☐ This is what binary search does in code.

🔄 Steps (Algorithm)

1. Start with the `left` and `right` pointers at the start and end of the list.
 2. While `left` is less than or equal to `right`:
 - Calculate the middle index.
 - Compare the target with the middle element:
 - If equal → Found the item.
 - If `target < middle` → Search in the **left half**.
 - If `target > middle` → Search in the **right half**.
 3. If the item is not found, return `-1` or "Not Found".
-

☐ Time & Space Complexity

Case	Complexity
------	------------

Best Case	O(1) — Found in the middle
-----------	-----------------------------------

Worst Case	O(log n) — Keep dividing
------------	---------------------------------

Case	Complexity
Space	O(1) — No extra space

🔗 Python Code with Detailed Comments

```
def binary_search(arr, target):
    # Step 1: Initialize pointers
    left = 0           # Start of the list
    right = len(arr) - 1 # End of the list

    # Step 2: Repeat the loop until left crosses right
    while left <= right:
        # Step 3: Find the middle index
        mid = (left + right) // 2

        # Step 4: Check the value at mid
        if arr[mid] == target:
            return mid # Found the target

        # Step 5: Decide where to search next
        elif target < arr[mid]:
            right = mid - 1
        else:
            left = mid + 1

    # Step 6: If not found, return -1
    return -1
```

🔗 Example

```
numbers = [2, 4, 6, 8, 10, 12, 14] # List must be sorted
target = 10

result = binary_search(numbers, target)

if result != -1:
    print(f"Element found at index {result}")
else:
    print("Element not found")
```

Output:

Element found at index 4

☐ Dry Run (Step-by-Step)

```
arr = [2, 4, 6, 8, 10], target = 8
left = 0, right = 4

mid = (0 + 4) // 2 = 2 → arr[mid] = 6

8 > 6 → search right → left = 3
```

```
mid = (3 + 4) // 2 = 3 → arr[mid] = 8  
8 == 8 → FOUND!
```

⊗ What if the list is unsorted?

```
arr = [10, 4, 6, 2, 8] # Not sorted
```

Binary search will give **wrong results**.

✓ Always sort the list first!

★ Bonus: Recursive Version in Python

```
def binary_search_recursive(arr, target, left, right):  
    if left > right:  
        return -1  
  
    mid = (left + right) // 2  
  
    if arr[mid] == target:  
        return mid  
    elif target < arr[mid]:  
        return binary_search_recursive(arr, target, left, mid - 1)  
    else:  
        return binary_search_recursive(arr, target, mid + 1, right)  
  
# Example  
arr = [1, 3, 5, 7, 9]  
print(binary_search_recursive(arr, 7, 0, len(arr) - 1)) # Output: 3
```

✓ Summary

Step	Action
1	Check if the list is sorted
2	Set left and right pointers
3	Loop until <code>left <= right</code>
4	Check middle value
5	Narrow the search (left/right half)
6	Return index if found or -1 if not

♣ What is DFS (Depth First Search)?

DFS is a graph traversal algorithm that starts from a source node and explores as far as possible along each branch before backtracking.

It uses a **stack data structure** (either explicitly or via recursion) and is commonly used in:

- ☐ Solving mazes
 - ☐ Path finding
 - 🔁 Topological sorting
 - 🔁 Cycle detection
 - ☐ Solving puzzles (e.g., Sudoku)
-

★ Key Concepts

- **Graph:** A collection of nodes (vertices) connected by edges.
 - DFS works on both **directed** and **undirected** graphs.
 - DFS can be implemented using:
 - **Recursion**
 - **Explicit Stack**
-

🔄 DFS Process (Step-by-Step)

1. Start at a node (called source).
 2. Mark the node as visited.
 3. For each unvisited neighbor:
 - Recursively apply DFS.
 4. Backtrack when no unvisited neighbor is left.
-

✓ DFS Pseudocode (Recursive)

```
DFS (node) :  
    if node is not visited:  
        mark node as visited  
        for each neighbor of node:  
            DFS (neighbor)
```

☐ DFS vs BFS (Key Differences)

Feature	DFS	BFS
Data Structure	Stack / Recursion	Queue
Explores	As deep as possible	Level by level
Time Complexity	$O(V + E)$	$O(V + E)$
Space Complexity	$O(V)$ (recursion/stack)	$O(V)$ (queue)
Use Case	Pathfinding, Cycle Detection Shortest path in unweighted graphs	

DFS Code in Python

1 Recursive DFS for Adjacency List Graph

```
def dfs(graph, node, visited):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbor in graph[node]:
            dfs(graph, neighbor, visited)

# Example graph (undirected)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

visited = set()
dfs(graph, 'A', visited)
# Output: A B D E F C (order may vary based on graph structure)
```

2 Iterative DFS Using Stack

```
def dfs_iterative(graph, start):
    visited = set()
    stack = [start]

    while stack:
        node = stack.pop()
        if node not in visited:
            print(node, end=" ")
            visited.add(node)
            # Push neighbors in reverse to maintain order
            for neighbor in reversed(graph[node]):
                stack.append(neighbor)

dfs_iterative(graph, 'A')
```

Time and Space Complexity

Let:

- V = number of vertices
- E = number of edges

Operation Complexity

Time $O(V + E)$

Space $O(V)$

☐ Applications of DFS

- ✓ Pathfinding in mazes
 - ✓ Topological Sorting (in DAGs)
 - ✓ Connected Components
 - ✓ Cycle Detection in Graphs
 - ✓ Solving puzzles (e.g., Sudoku)
 - ✓ Web Crawlers (Explore all linked pages)
-

☐ Real-world Analogy

Imagine a maze. DFS keeps going down one path until it can't go any further, then backtracks and tries another.
It's like exploring **every hallway deeply** before trying the next one.

🔍 What is Breadth-First Search (BFS)?

BFS is a graph traversal algorithm. It starts at a starting node (called the **root** in tree structures) and explores all the neighboring nodes at the current depth before moving on to the nodes at the next depth level.

🐘 **Think of it as:**

Waves of exploration — first explore all close nodes, then go deeper step by step.

✓ **It works for:**

- Graphs (connected or disconnected)
 - Trees (BFS in trees = Level Order Traversal)
-

☐ Core Concepts

∞ **Graph Terminology**

- **Node (or Vertex):** A point in the graph.
 - **Edge:** A connection between two nodes.
 - **Neighbor:** A node connected directly to another node.
-

☐ What BFS Needs:

- **Queue:** To process nodes in "First In First Out" (FIFO) order.
 - **Visited Set:** To keep track of already visited nodes (avoid infinite loops).
-

🔑 BFS Strategy (Step-by-step)

1. Start from the source node.
2. Mark it as visited.
3. Put it in a queue.
4. While the queue is not empty:
 - Remove the node from the front.
 - For each unvisited neighbor:

- Mark it as visited
- Add it to the queue

🔗 BFS Implementation in Python (from Scratch)

We'll represent the graph as an **adjacency list** using a dictionary.

Example Graph:

```
A -- B -- D
|      |
C      E
```

Adjacency list:

```
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A'],
    'D': ['B'],
    'E': ['B']
}
```

Python Code:

```
from collections import deque # for efficient queue

def bfs(graph, start_node):
    visited = set()           # To keep track of visited nodes
    queue = deque([start_node]) # Start with the starting node

    while queue:
        node = queue.popleft() # Remove node from the front of the queue

        if node not in visited:
            print("Visited:", node)
            visited.add(node)    # Mark the node as visited

            # Add all unvisited neighbors to the queue
            for neighbor in graph[node]:
                if neighbor not in visited:
                    queue.append(neighbor)

# Run the BFS:
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A'],
    'D': ['B'],
    'E': ['B']
}

bfs(graph, 'A')
```

Output:

Visited: A
Visited: B
Visited: C
Visited: D
Visited: E

 How BFS Works (Trace)

- Start at A → Queue: ['B', 'C']
 - Visit B → Queue: ['C', 'D', 'E']
 - Visit C → Queue: ['D', 'E']
 - Visit D → Queue: ['E']
 - Visit E → Queue: [] (done!)
-

 When to Use BFS?

- ✓ Find **shortest path** in unweighted graphs
 - ✓ **Level order traversal** of trees
 - ✓ Web crawling
 - ✓ Social networking (shortest connection path)
 - ✓ Maze solving
-

 Key Points Summary

Concept	Description
Type	Traversal Algorithm
Data Structure	Queue (FIFO)
Time Complexity	$O(V + E) \rightarrow V$: Vertices, E : Edges
Space Complexity	$O(V)$ for visited set and queue
Use Case	Shortest path, Level order, Connectivity

Insertion Sort in Python – Detailed Explanation

◆ What is Insertion Sort?

Insertion Sort is a **simple sorting algorithm** inspired by how we arrange playing cards in our hand.

- Imagine picking cards one by one and inserting each card in its correct position relative to already sorted cards in your hand.

Steps:

1. Start from the **second element** (because a single element is already sorted).
2. Compare it with the elements before it.
3. Shift the elements greater than the current element to the right.
4. Insert the current element in the correct position.
5. Repeat until the list is sorted.

◆ Insertion Sort Algorithm in Python

```
def insertion_sort(arr):
    # Traverse from the second element to the end
    for i in range(1, len(arr)):
        key = arr[i]    # current element to insert
        j = i - 1       # index of the previous element

        # Move elements greater than key one position ahead
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        # Insert the key in its correct position
        arr[j + 1] = key

    return arr

# Example usage
data = [12, 11, 13, 5, 6]
print("Original:", data)
print("Sorted:", insertion_sort(data))
```

✓ Output:

```
Original: [12, 11, 13, 5, 6]
Sorted: [5, 6, 11, 12, 13]
```

◆ Dry Run Example

Input: [12, 11, 13, 5, 6]

- Pass 1: [11, 12, 13, 5, 6]
 - Pass 2: [11, 12, 13, 5, 6] (already in correct place)
 - Pass 3: [5, 11, 12, 13, 6]
 - Pass 4: [5, 6, 11, 12, 13]
-

◆ Time Complexity of Insertion Sort

- **Best Case (Already Sorted):** $O(n)$
 - **Worst Case (Reverse Sorted):** $O(n^2)$
 - **Average Case:** $O(n^2)$
-

◆ Space Complexity

- $O(1) \rightarrow$ In-place sorting (uses only a few extra variables).
-

◆ Comparison with Other Sorting Algorithms

Algorithm	Best Case	Average Case	Worst Case	Space Complexity	Stable?
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓ Yes
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓ Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✗ No
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓ Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	✗ No (depends)
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	✗ No

◆ Why use Insertion Sort?

- Simple and easy to implement.
- Efficient for **small arrays** or **partially sorted arrays**.
- Useful in real-time when new data keeps coming in (**online sorting**).
- Not suitable for large datasets because of $O(n^2)$ time complexity.

Heap Sort – Explanation and Code

◆ What is Heap Sort?

Heap Sort is a **comparison-based sorting algorithm** that uses a **Binary Heap data structure** (usually a Max Heap) to sort elements.

It works in **two main steps**:

1. **Build a Max Heap** from the array.
 - A Max Heap is a complete binary tree where each node is greater than or equal to its children.
 - This ensures the **largest element is always at the root**.
2. **Extract elements from the heap one by one**:
 - Swap the root (largest element) with the last element in the heap.
 - Reduce the heap size by 1.
 - Heapify the root again to restore the heap property.
 - Repeat until the array is sorted.

◆ Why Heap Sort?

- **Time Complexity**:
 - Best Case: $O(n \log n)$
 - Average Case: $O(n \log n)$
 - Worst Case: $O(n \log n)$
- **Space Complexity**: $O(1)$ (in-place sorting).
- **Not a stable sort** (does not preserve the order of equal elements).

◆ The Heapify Process

Heapify is the **core operation** in Heap Sort.

Steps in Heapify:

1. Assume index i is the root.
2. Find indices of left child $= 2*i+1$ and right child $= 2*i+2$.
3. Compare root with its children.
4. Swap root with the largest child if needed.
5. Recursively heapify the affected subtree.

This ensures that the **subtree rooted at i satisfies the heap property**.

◆ Heap Sort Algorithm (Step by Step)

1. **Build a Max Heap** from the given array (call heapify for all non-leaf nodes).
 2. **Swap the root with the last element.**
 3. **Heapify the reduced heap.**
 4. **Repeat** until the entire array is sorted.
-

◆ Python Code for Heap Sort

```
def heapify(arr, n, i):
    """Ensure the subtree rooted at index i is a max heap."""
    largest = i          # Assume root is largest
    left = 2 * i + 1     # left child
    right = 2 * i + 2    # right child

    # If left child exists and is greater than root
    if left < n and arr[left] > arr[largest]:
        largest = left

    # If right child exists and is greater than largest so far
    if right < n and arr[right] > arr[largest]:
        largest = right

    # If root is not largest, swap and continue heapifying
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest) # Recursive heapify

def heap_sort(arr):
    n = len(arr)

    # Step 1: Build a max heap
    for i in range(n // 2 - 1, -1, -1): # Start from last non-leaf node
        heapify(arr, n, i)

    # Step 2: Extract elements from heap one by one
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0] # Swap root with last element
        heapify(arr, i, 0)              # Heapify reduced heap

# Example usage
arr = [12, 11, 13, 5, 6, 7]
print("Original array:", arr)
heap_sort(arr)
print("Sorted array:", arr)
```

◆ Example Walkthrough

Take the array:

```
arr = [12, 11, 13, 5, 6, 7]
```

1. **Build Max Heap**
→ [13, 11, 12, 5, 6, 7]
 2. **Extract 1st max (13)**
 - Swap root with last: [7, 11, 12, 5, 6, 13]
 - Heapify → [12, 11, 7, 5, 6, 13]
 3. **Extract 2nd max (12)**
 - Swap: [6, 11, 7, 5, 12, 13]
 - Heapify → [11, 6, 7, 5, 12, 13]
 4. Continue the process...
Final sorted array → [5, 6, 7, 11, 12, 13].
-

◆ Output

Original array: [12, 11, 13, 5, 6, 7]
Sorted array: [5, 6, 7, 11, 12, 13]

◆ Summary

- Heap Sort uses a **binary heap** to sort data.
- It is an **in-place sorting algorithm** with $O(1)$ extra space.
- It always runs in $O(n \log n)$ time, regardless of input.
- It is **not stable**, but it is **efficient** and widely used.

Selection Sort – Explanation and Code

◆ What is Selection Sort?

Selection Sort is one of the simplest **comparison-based sorting algorithms**.

It works by **repeatedly finding the smallest element** from the unsorted part of the array and moving it to the beginning.

◆ How Selection Sort Works (Step by Step)

1. Start with the first element (at index 0).
 2. Find the **minimum element** from the unsorted part of the array.
 3. Swap the minimum element with the element at the beginning of the unsorted part.
 4. Move the boundary of the sorted part forward by one.
 5. Repeat until the whole array is sorted.
-

◆ Why Selection Sort?

- **Time Complexity:**
 - Best Case: $O(n^2)$
 - Average Case: $O(n^2)$
 - Worst Case: $O(n^2)$
 - **Space Complexity:** $O(1)$ (in-place sorting).
 - **Not stable** (equal elements may lose their relative order).
 - **Very easy to understand**, but inefficient for large datasets compared to Heap Sort, Merge Sort, or Quick Sort.
-

◆ Selection Sort Algorithm

1. Loop i from 0 to $n-1$.
 2. Assume i is the index of the minimum element.
 3. Compare with every other element in the unsorted part.
 4. Update index of the minimum if a smaller element is found.
 5. Swap the found minimum with element at index i .
-

◆ Python Code for Selection Sort

```
def selection_sort(arr):
    n = len(arr)

    # Traverse through all elements
    for i in range(n):
        # Assume the minimum is the first element in the unsorted part
        min_index = i

        # Find the index of the minimum element in the unsorted part
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        # Swap the found minimum with the first element of the unsorted
part
        arr[i], arr[min_index] = arr[min_index], arr[i]

# Example usage
arr = [64, 25, 12, 22, 11]
print("Original array:", arr)
selection_sort(arr)
print("Sorted array:", arr)
```

◆ Example Walkthrough

Take the array:

[64, 25, 12, 22, 11]

1. **Pass 1:**
 - Minimum element = 11
 - Swap with first element → [11, 25, 12, 22, 64]
 2. **Pass 2:**
 - Minimum element = 12
 - Swap with second element → [11, 12, 25, 22, 64]
 3. **Pass 3:**
 - Minimum element = 22
 - Swap with third element → [11, 12, 22, 25, 64]
 4. **Pass 4:**
 - Minimum element = 25
 - Swap with fourth element → [11, 12, 22, 25, 64]
 5. **Pass 5:**
 - Last element already in place → [11, 12, 22, 25, 64]
-

◆ Output

Original array: [64, 25, 12, 22, 11]
Sorted array: [11, 12, 22, 25, 64]

◆ Summary

- Selection Sort repeatedly finds the minimum element and puts it in the correct position.
- It has **$O(n^2)$** time complexity regardless of input.
- It is **in-place** but **not stable**.
- Useful for **small datasets** or when simplicity is more important than performance.

Merge Sort – Explanation and Code

◆ What is Merge Sort?

Merge Sort is a **divide and conquer algorithm** used for sorting.

It works by **dividing the array into smaller sub-arrays**, sorting them recursively, and then **merging the sorted sub-arrays** to produce the final sorted array.

◆ How Merge Sort Works (Step by Step)

1. **Divide:** Split the array into two halves (left and right).
 2. **Conquer:** Recursively sort the two halves.
 3. **Combine:** Merge the two sorted halves into a single sorted array.
-

◆ Why Merge Sort?

- **Time Complexity:**
 - Best Case: $O(n \log n)$
 - Average Case: $O(n \log n)$
 - Worst Case: $O(n \log n)$
 - **Space Complexity:** $O(n)$ (requires extra memory for merging).
 - **Stable sort** (preserves the order of equal elements).
 - **Efficient for large datasets**, unlike Selection or Bubble Sort.
-

◆ Merge Sort Algorithm

1. If the array has **0 or 1 element**, it is already sorted.
 2. Otherwise:
 - Divide the array into two halves.
 - Recursively apply merge sort on both halves.
 - Merge the two sorted halves together.
-

◆ Python Code for Merge Sort

```
def merge_sort(arr):
    if len(arr) > 1:
        # Find the middle point
        mid = len(arr) // 2

        # Divide the array into two halves
        left_half = arr[:mid]
        right_half = arr[mid:]

        # Recursively sort both halves
        merge_sort(left_half)
        merge_sort(right_half)

        # Merge the two halves
        i = j = k = 0

        # Compare elements from both halves and merge
        while i < len(left_half) and j < len(right_half):
            if left_half[i] < right_half[j]:
                arr[k] = left_half[i]
                i += 1
            else:
                arr[k] = right_half[j]
                j += 1
            k += 1
```

```
# Copy remaining elements of left_half, if any
while i < len(left_half):
    arr[k] = left_half[i]
    i += 1
    k += 1

# Copy remaining elements of right_half, if any
while j < len(right_half):
    arr[k] = right_half[j]
    j += 1
    k += 1

# Example usage
arr = [38, 27, 43, 3, 9, 82, 10]
print("Original array:", arr)
merge_sort(arr)
print("Sorted array:", arr)
```

◆ Example Walkthrough

Take the array:

[38, 27, 43, 3, 9, 82, 10]

1. **Divide Step:**
 - Split into [38, 27, 43] and [3, 9, 82, 10].
 - Keep splitting until single elements remain.
 2. **Sorting Single Elements (Base Case):**
 - [38] [27] [43] [3] [9] [82] [10]
 3. **Merge Step:**
 - Merge [38] and [27] → [27, 38]
 - Merge [27, 38] and [43] → [27, 38, 43]
 - Merge [3] and [9] → [3, 9]
 - Merge [3, 9] and [82] → [3, 9, 82]
 - Merge [3, 9, 82] and [10] → [3, 9, 10, 82]
 4. **Final Merge:**
 - Merge [27, 38, 43] and [3, 9, 10, 82] → [3, 9, 10, 27, 38, 43, 82]
-

◆ Output

Original array: [38, 27, 43, 3, 9, 82, 10]
Sorted array: [3, 9, 10, 27, 38, 43, 82]

◆ Summary

- Merge Sort uses **divide and conquer** to sort arrays.

- Runs in $O(n \log n)$ time for all cases.
 - Requires $O(n)$ extra space.
 - It is a **stable sort**, making it useful in scenarios where the relative order of equal elements matters.
 - Works well for **large datasets** and **linked lists**.
-

Quick Sort – Explanation and Code

◆ What is Quick Sort?

Quick Sort is a **divide and conquer sorting algorithm** that works by selecting a **pivot element**, partitioning the array around the pivot, and then recursively sorting the sub-arrays.

It is one of the fastest sorting algorithms in practice and is widely used in real-world applications.

◆ How Quick Sort Works (Step by Step)

1. **Choose a Pivot:** Pick an element as the pivot (commonly the last element, but other strategies exist).
 2. **Partition:** Rearrange elements so that:
 - Elements **smaller than the pivot** go to the left.
 - Elements **greater than the pivot** go to the right.
 3. **Recursively Apply:** Apply Quick Sort on the left and right sub-arrays.
 4. **Combine:** Since sorting is done in place, the array becomes sorted when recursion ends.
-

◆ Why Quick Sort?

- **Time Complexity:**
 - Best Case: $O(n \log n)$ (balanced partitions).
 - Average Case: $O(n \log n)$
 - Worst Case: $O(n^2)$ (if pivot choice is poor, e.g., sorted array with last element as pivot).
 - **Space Complexity:** $O(\log n)$ (due to recursion stack).
 - **Not stable** (relative order of equal elements may change).
 - **Very efficient in practice** because of good cache performance and fewer comparisons.
-

◆ Quick Sort Algorithm

1. Pick a pivot element.

2. Partition the array into two sub-arrays: elements smaller than pivot, elements larger than pivot.
 3. Recursively apply Quick Sort on the sub-arrays.
 4. Combine results (in-place).
-

◆ Python Code for Quick Sort

```
def partition(arr, low, high):
    # Choose the last element as pivot
    pivot = arr[high]
    i = low - 1 # index of smaller element

    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i] # swap if element <= pivot

    # Place pivot at the correct position
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

def quick_sort(arr, low, high):
    if low < high:
        # Partition index
        pi = partition(arr, low, high)

        # Recursively sort elements before and after partition
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

# Example usage
arr = [10, 7, 8, 9, 1, 5]
print("Original array:", arr)
quick_sort(arr, 0, len(arr) - 1)
print("Sorted array:", arr)
```

◆ Example Walkthrough

Take the array:

[10, 7, 8, 9, 1, 5]

1. **First Partition:**
 - Pivot = 5
 - Rearranged → [1, 5, 8, 9, 10, 7]
 - Pivot 5 is placed in correct position (index 1).
2. **Left Sub-array:** [1] → already sorted.
3. **Right Sub-array:** [8, 9, 10, 7]
 - Pivot = 7

- Rearranged $\rightarrow [7, 9, 10, 8]$
 - Pivot 7 at correct position.
4. Continue partitioning until all sub-arrays are of size 1.

Final sorted array $\rightarrow [1, 5, 7, 8, 9, 10]$.

◆ Output

Original array: [10, 7, 8, 9, 1, 5]
Sorted array: [1, 5, 7, 8, 9, 10]

◆ Summary

- Quick Sort uses **divide and conquer** with partitioning around a pivot.
 - Average case runs in $O(n \log n)$, but worst case is $O(n^2)$.
 - Performs better than Merge Sort in practice (due to lower memory overhead).
 - **In-place** but **not stable**.
 - One of the **most widely used sorting algorithms** in libraries and systems.
-

Counting Sort – Explanation and Code

◆ What is Counting Sort?

Counting Sort is a **non-comparison-based sorting algorithm**.

Instead of comparing elements (like Quick Sort or Merge Sort), it works by **counting the number of occurrences** of each element in the input.

It is mainly useful when:

- The input elements are **integers**.
 - The range of elements ($\text{max} - \text{min}$) is **not significantly larger than the number of elements**.
-

◆ How Counting Sort Works (Step by Step)

1. **Find the maximum element** in the array (let's call it k).
 2. **Create a count array** of size $k+1$, initialized with zeros.
 3. **Count each element's frequency** by iterating through the input array.
 4. **Update count array** to store cumulative counts (this tells us positions of elements in the sorted array).
 5. **Build the output array** by placing elements in their correct positions using the count array.
 6. Copy the output array back to the original array.
-

◆ Why Counting Sort?

- **Time Complexity:**
 - Best Case: $O(n + k)$
 - Average Case: $O(n + k)$
 - Worst Case: $O(n + k)$
 - **Space Complexity:** $O(n + k)$ (extra space needed for count and output arrays).
 - **Stable sort** (if implemented properly).
 - **Efficient for small ranges**, but **not suitable for large ranges** (e.g., sorting numbers up to billions).
-

◆ Counting Sort Algorithm

1. Find the maximum element in the array.
 2. Initialize a count array of that size.
 3. Count occurrences of each element.
 4. Compute cumulative sums to determine positions.
 5. Build the output array.
 6. Copy it back to the original.
-

◆ Python Code for Counting Sort

```
def counting_sort(arr):
    # Step 1: Find the maximum element
    max_val = max(arr)

    # Step 2: Initialize count array
    count = [0] * (max_val + 1)

    # Step 3: Count the occurrences of each element
    for num in arr:
        count[num] += 1

    # Step 4: Update count[i] to store cumulative sum
    for i in range(1, len(count)):
        count[i] += count[i - 1]

    # Step 5: Build the output array (stable)
    output = [0] * len(arr)
    for num in reversed(arr): # reversed to maintain stability
        output[count[num] - 1] = num
        count[num] -= 1

    # Step 6: Copy sorted elements back into original array
    for i in range(len(arr)):
        arr[i] = output[i]

# Example usage
arr = [4, 2, 2, 8, 3, 3, 1]
print("Original array:", arr)
counting_sort(arr)
print("Sorted array:", arr)
```

◆ Example Walkthrough

Take the array:

[4, 2, 2, 8, 3, 3, 1]

1. **Find max element:** 8
→ Create count array of size 9 → [0, 0, 0, 0, 0, 0, 0, 0, 0]

2. **Count occurrences:**

After counting $\rightarrow [0, 1, 2, 2, 1, 0, 0, 0, 1]$
(meaning: one 1, two 2s, two 3s, one 4, one 8)

3. **Cumulative sum:**

$\rightarrow [0, 1, 3, 5, 6, 6, 6, 6, 7]$
(now each index tells us the final position of that number).

4. **Build output (placing elements in correct order):**

$\rightarrow [1, 2, 2, 3, 3, 4, 8]$

◆ Output

Original array: `[4, 2, 2, 8, 3, 3, 1]`
Sorted array: `[1, 2, 2, 3, 3, 4, 8]`

◆ Summary

- Counting Sort is a **non-comparison sorting algorithm**.
 - Runs in $O(n + k)$ time, where k is the range of input values.
 - Requires extra memory ($O(n + k)$), so it is **not memory efficient for large ranges**.
 - **Stable sort**, which makes it suitable for sorting objects by multiple keys.
 - Best for integers with a **small range** relative to the array size.
-

Bubble Sort – Explanation and Code

◆ What is Bubble Sort?

Bubble Sort is the **simplest comparison-based sorting algorithm**.

It repeatedly **swaps adjacent elements** if they are in the wrong order. After each pass, the largest element "bubbles up" to its correct position at the end of the array.

◆ How Bubble Sort Works (Step by Step)

1. Start from the beginning of the array.
 2. Compare the first two adjacent elements.
 - If the first is greater than the second → swap them.
 3. Move to the next pair, and repeat until the end of the array.
 4. After the first pass, the **largest element is at the end**.
 5. Repeat the process for the remaining elements (ignoring the last sorted ones).
 6. Continue until no swaps are needed → array is sorted.
-

◆ Why Bubble Sort?

- **Time Complexity:**
 - Best Case: $O(n)$ (if array already sorted, with optimization).
 - Average Case: $O(n^2)$
 - Worst Case: $O(n^2)$
 - **Space Complexity:** $O(1)$ (in-place sorting).
 - **Stable sort** (preserves the order of equal elements).
 - **Easy to understand**, but very inefficient for large datasets.
-

◆ Bubble Sort Algorithm

1. Loop i from 0 to $n-1$.
 2. For each pass, loop j from 0 to $n-i-1$.
 3. Compare adjacent elements `arr[j]` and `arr[j+1]`.
 4. Swap if they are in the wrong order.
 5. Stop early if no swaps happen in a pass (optimization).
-

◆ Python Code for Bubble Sort

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        swapped = False # Optimization: check if any swap happens

        # Last i elements are already in place
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        # If no swaps happened → array is already sorted
        if not swapped:
            break

# Example usage
arr = [64, 34, 25, 12, 22, 11, 90]
print("Original array:", arr)
bubble_sort(arr)
print("Sorted array:", arr)
```

◆ Example Walkthrough

Take the array:

[64, 34, 25, 12, 22, 11, 90]

1. **Pass 1:**
After comparing/swapping → [34, 25, 12, 22, 11, 64, 90]
(largest 90 is bubbled to the end).
2. **Pass 2:**
→ [25, 12, 22, 11, 34, 64, 90]
3. **Pass 3:**
→ [12, 22, 11, 25, 34, 64, 90]
4. **Pass 4:**
→ [12, 11, 22, 25, 34, 64, 90]
5. **Pass 5:**
→ [11, 12, 22, 25, 34, 64, 90]

Array is now sorted.

◆ Output

Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array: [11, 12, 22, 25, 34, 64, 90]

◆ Summary

- Bubble Sort works by **repeatedly swapping adjacent elements** until the array is sorted.
 - It is **stable** and **in-place**, but very inefficient ($O(n^2)$).
 - Useful only for **small datasets or teaching purposes**.
 - Rarely used in practice compared to **Merge Sort, Quick Sort, or Heap Sort**.
-

Kruskal's Algorithm — Explanation and Python Implementation

Short Summary

Kruskal's algorithm finds a **Minimum Spanning Tree (MST)** of a connected, undirected, weighted graph.

It is a **greedy algorithm** that repeatedly picks the smallest-weight edge that doesn't create a cycle, until every vertex is connected (or until no more safe edges exist, producing a **minimum spanning forest** for disconnected graphs).

Why It's Used

- To build a lowest-cost set of connections that links all nodes (telecom, roads, electric grids).
 - It's conceptually simple and easy to implement when you can list all edges.
 - Works well for **sparse graphs** (E much smaller than V^2).
 - Produces the global optimal MST thanks to greedy correctness.
-

When to Use (vs Alternatives)

- Use **Kruskal** when you have an **edge list** or the graph is sparse.
 - Use **Prim** when you have adjacency lists and want to start from a vertex (Prim with a binary heap is efficient).
 - If the graph is **directed**, MST is not defined — you'd need directed-arborescence algorithms.
-

High-Level Algorithm

1. Sort all edges by weight (ascending).
 2. Initialize a **Disjoint Set Union (Union-Find)** structure for vertices.
 3. Iterate edges in ascending order; for each edge (u, v, w) :
 - If `find(u) != find(v)`, include the edge and `union(u, v)`.
 - Else skip the edge (would form a cycle).
 4. Stop when you have $V-1$ edges (for a connected graph).
-

Time & Space Complexity

- Sorting edges: $O(E \log E) \approx O(E \log V)$.
- Union-Find operations: nearly $O(1)$ each (with path compression + union by rank).
- **Total time:** $O(E \log V)$.
- **Space:** $O(E + V)$.

Kruskal performs especially well when the graph is **sparse**.

Union-Find (Disjoint Set)

- Supports `find(x)` and `union(x, y)`.
 - Optimizations: **path compression** + **union by rank** → amortized near-constant time.
 - Used to quickly test whether adding an edge will create a cycle.
-

Python Implementation

```
from typing import Iterable, Tuple, List, Any

class UnionFind:
    def __init__(self, elements: Iterable[Any]):
        self.parent = {x: x for x in elements}
        self.rank = {x: 0 for x in elements}

    def find(self, x: Any) -> Any:
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, a: Any, b: Any) -> bool:
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.rank[ra] < self.rank[rb]:
            self.parent[ra] = rb
        elif self.rank[ra] > self.rank[rb]:
            self.parent[rb] = ra
        else:
            self.parent[rb] = ra
            self.rank[ra] += 1
        return True

def kruskal(edges: Iterable[Tuple[Any, Any, float]]) ->
    Tuple[List[Tuple[Any, Any, float]], float]:
    edges_list = list(edges)
    nodes = set()
    for u, v, _w in edges_list:
        nodes.add(u); nodes.add(v)
    edges_list.sort(key=lambda e: e[2])

    uf = UnionFind(nodes)
    mst = []
    total_weight = 0.0
```

```
for u, v, w in edges_list:
    if uf.union(u, v):
        mst.append((u, v, w))
        total_weight += w
        if len(mst) == len(nodes) - 1:
            break
return mst, total_weight

if __name__ == "__main__":
    edges = [
        ('A', 'B', 4), ('A', 'C', 3), ('B', 'C', 1),
        ('B', 'D', 2), ('C', 'D', 4), ('C', 'E', 2), ('D', 'E', 3)
    ]
    mst_edges, weight = kruskal(edges)
    print("MST edges:")
    for u, v, w in mst_edges:
        print(f"{u} - {v} : {w}")
    print("Total MST weight:", weight)
```

Example Run Output

```
MST edges:
B - C : 1
B - D : 2
C - E : 2
A - C : 3
Total MST weight: 8.0
```

Use Cases

- **Telecommunications / network layout** (minimize cable cost).
 - **Road planning, utility distribution** (power lines).
 - **Clustering** (single-linkage hierarchical clustering uses MST).
 - **Image segmentation** techniques.
 - Producing baseline **low-cost connection skeletons** for more complex optimization.
-

Tips & Notes

- Works only on **undirected graphs with weights**.
 - If graph is **disconnected** → produces a **minimum spanning forest**.
 - Handles **negative weights** (still works).
 - For **dense graphs**, Prim may be more efficient.
-

Dijkstra's Algorithm — Explanation + Python implementation

Short summary

Dijkstra's algorithm computes the **shortest path distances** from a single source node to every other node in a graph with **non-negative edge weights**. It's a greedy algorithm that repeatedly selects the currently closest (unsettled) node and relaxes its outgoing edges using a priority queue (min-heap).

Why it's used

- To find the cheapest/shortest route from one place to many places (single-source shortest paths) in networks with **non-negative** weights.
 - It's efficient and simple to implement with a binary heap.
 - Widely used in routing, mapping, and any system that needs shortest-paths quickly on graphs with non-negative costs.
-

When to use / limitations

- **Use Dijkstra** when:
 - Edge weights are **non-negative**.
 - You need shortest paths from a single source to all (or some) nodes.
 - Graphs can be directed or undirected.
 - **Don't use** Dijkstra if edges can have **negative weights** → use **Bellman–Ford** (or detect and fix negatives).
 - For **multiple sources** / **all pairs** shortest paths, consider Floyd–Warshall (dense small graphs) or Johnson's algorithm (sparse graphs).
-

Time & space complexity

- With a binary heap (Python `heapq`) and adjacency list:
 - **Time:** $O((V + E) \log V)$ which is commonly written as **$O(E \log V)$** for connected graphs (each edge may cause a heap push, and each push/pop costs $O(\log V)$).
 - **Space:** $O(V + E)$ to store the graph (adjacency list) plus $O(V)$ for distance/prev/heap.
- With a naive array (no heap): **$O(V^2)$** time (better for dense graphs or small V).
- With a Fibonacci heap (rare in practice): **$O(E + V \log V)$** .

Python implementation (readable, robust — works for arbitrary hashable nodes)

```
import heapq
from typing import Dict, List, Tuple, Any, Optional

def dijkstra(graph: Dict[Any, List[Tuple[Any, float]]], source: Any):
    """
    Single-source shortest paths using Dijkstra (non-negative weights).

    Args:
        graph: adjacency dict where graph[u] = list of (v, weight)
              (graph can be directed or undirected).
        source: starting node (must be present or at least referenced).

    Returns:
        dist: dict mapping node -> shortest distance from source
              (float('inf') if unreachable)
        prev: dict mapping node -> predecessor on shortest path (None if no
              predecessor)
    """
    # Collect all nodes (in case some appear only as neighbors)
    nodes = set(graph.keys())
    for nbrs in graph.values():
        for v, _w in nbrs:
            nodes.add(v)

    dist: Dict[Any, float] = {node: float('inf') for node in nodes}
    prev: Dict[Any, Optional[Any]] = {node: None for node in nodes}

    dist[source] = 0.0
    # heap of (distance_to_node, node)
    heap: List[Tuple[float, Any]] = [(0.0, source)]

    while heap:
        d_u, u = heapq.heappop(heap)
        # If we popped a stale (outdated) pair, skip it
        if d_u > dist[u]:
            continue

        # Relax neighbors
        for v, weight in graph.get(u, []):
            if weight < 0:
                raise ValueError("Dijkstra cannot handle negative edge weights")

            alt = d_u + weight
            if alt < dist[v]:
                dist[v] = alt
                prev[v] = u
                heapq.heappush(heap, (alt, v))

    return dist, prev


def reconstruct_path(prev: Dict[Any, Optional[Any]], source: Any, target: Any) -> List[Any]:
    """Reconstruct shortest path from source to target using prev dict.
```

```

    Returns an empty list if target unreachable."""
    path = []
    node = target
    while node is not None:
        path.append(node)
        if node == source:
            break
        node = prev.get(node)
    path.reverse()
    if not path or path[0] != source:
        return [] # unreachable
    return path

# Example usage
if __name__ == "__main__":
    # Example directed graph (weights non-negative)
    graph = {
        'A': [('B', 4), ('C', 2)],
        'B': [('C', 5), ('D', 10)],
        'C': [('E', 3)],
        'D': [('F', 11)],
        'E': [('D', 4)],
        # 'F': [] # optional: nodes can be included implicitly
    }

    dist, prev = dijkstra(graph, 'A')
    print("Distances from A:")
    for node in sorted(dist):
        print(f"    {node:>2}: {dist[node]}")

    # Reconstruct path A -> F
    path = reconstruct_path(prev, 'A', 'F')
    print("\nShortest path A -> F:", path)

```

Line-by-line / block-by-block explanation

1. Graph representation

- graph is a dictionary keyed by node; each value is a list of (neighbor, weight) tuples.
- This adjacency-list form is memory-efficient for sparse graphs.

2. Preparing nodes & init

- We build a `nodes` set because sometimes nodes only appear as neighbors and may be missing as keys.
- `dist[node] = float('inf')` initially meaning unknown/very large distance.
- `prev[node] = None` will store the predecessor used to reconstruct the shortest path.

3. Heap (priority queue)

- `heapq` holds `(distance_so_far, node)` tuples. The heap gives the next node with smallest provisional distance.
- We push the source with distance `0.0`.

4. Main loop

- Pop the smallest-distance tuple `(d_u, u)`.

- **Stale check:** If $d_u > \text{dist}[u]$ skip — this entry is outdated because we already found a shorter path to u and pushed a newer tuple.
- For every neighbor (v, weight) :
 - Compute alternative distance $\text{alt} = d_u + \text{weight}$.
 - If $\text{alt} < \text{dist}[v]$, update $\text{dist}[v]$ and set $\text{prev}[v] = u$. Push new (alt, v) onto the heap.
- Repeat until heap empty (all reachable nodes settled).
- 5. **Path reconstruction**
 - Follow prev pointers from the target back to the source, reverse the list. If the path doesn't start with the source, the target is unreachable.
- 6. **Negative weights**
 - Dijkstra assumes weights are non-negative. If a negative weight is encountered, raise error or use Bellman-Ford instead.

Step-by-step (short trace on the example)

Given graph above and source 'A':

1. Init: $\text{dist}[A]=0$, others inf . Heap = $[(0, A)]$.
2. Pop $(0, A)$, relax B $\rightarrow \text{dist}[B]=4$, $\text{prev}[B]=A$; relax C $\rightarrow \text{dist}[C]=2$, $\text{prev}[C]=A$. Heap now $[(2, C), (4, B)]$.
3. Pop $(2, C)$, relax E $\rightarrow \text{dist}[E]=5$, $\text{prev}[E]=C$. Heap now $[(4, B), (5, E)]$.
4. Pop $(4, B)$, relax D $\rightarrow \text{dist}[D]=14$, $\text{prev}[D]=B$. Heap now $[(5, E), (14, D)]$.
5. Pop $(5, E)$, relax D $\rightarrow \text{alt}=9 < 14$, so $\text{dist}[D]=9$, $\text{prev}[D]=E$; push $(9, D)$. Heap $[(9, D), (14, D)]$.
6. Pop $(9, D)$ (14,D is stale), relax F $\rightarrow \text{dist}[F]=20$, $\text{prev}[F]=D$.
7. Continue until heap empty. Final distances: A:0, B:4, C:2, E:5, D:9, F:20.

Reconstructing A $\rightarrow$ F gives ['A', 'C', 'E', 'D', 'F'].

Use cases

- **Map routing / navigation** (shortest driving/walking distances when costs non-negative).
- **Network routing** (shortest path in link-state protocols).
- **Game AI pathfinding** (often combined with heuristics $\rightarrow A^*$).
- **Resource allocation / logistics** where costs are additive and non-negative.
- **Shortest cost in weighted directed graphs** (when all weights are non-negative).

Variants & tips

- To find **shortest path to a single target** faster, stop the algorithm when the target node is popped from the heap (it's then finalized).
 - For **dense graphs** ($E \approx V^2$), the $O(V^2)$ array-based implementation can be faster in practice.
 - For **all pairs** shortest paths:
 - Use **Floyd–Warshall** for small dense graphs.
 - Use **Johnson's algorithm** (runs Dijkstra from each vertex with weight reweighting) for sparse graphs.
 - For better theoretical bounds with many decrease-key operations, Fibonacci heaps give $O(E + V \log V)$, but they are rarely used in production (complex).
 - If you need integer weights and very fast performance, specialized algorithms (e.g., Dial's algorithm / bucket queues) may be preferable.
-

Bellman–Ford Algorithm — Explanation + Python implementation

Short summary

Bellman–Ford computes **single-source shortest paths** in a graph that **may have negative edge weights**. It also **detects negative-weight cycles** reachable from the source (which make shortest paths undefined). The algorithm relaxes all edges repeatedly ($|V|-1$ times), and then makes one extra pass to check for further relaxations (which indicate negative cycles).

Why it's used

- It works when edge weights can be **negative** (Dijkstra cannot handle negative weights).
 - It **detects negative cycles** reachable from the source and can report which nodes are affected.
 - It's simple and works for both **directed** and **undirected** graphs (treat undirected edges as two directed edges).
-

When to use (vs alternatives)

- **Use Bellman–Ford** when: you need shortest paths and the graph may contain **negative edge weights**, or you need to detect negative cycles.
 - **Use Dijkstra** when all weights are **non-negative** — it's faster ($O(E \log V)$).
 - For **all pairs** shortest paths: consider **Johnson's algorithm** (reweights edges using Bellman–Ford, then run Dijkstra from every vertex) or **Floyd–Warshall** for dense graphs.
-

Time & space complexity

- **Time:** $O(V * E)$ in the worst case ($|V|$ vertices, $|E|$ edges).
 - **Space:** $O(V + E)$ to store graph plus $O(V)$ for `dist / prev` arrays/dicts. Bellman–Ford is significantly slower than Dijkstra for large graphs without negative weights, but necessary when negatives exist.
-

High-level algorithm

1. Initialize $\text{dist}[\text{source}] = 0$ and $\text{dist}[\text{other}] = +\infty$. $\text{prev}[\text{node}] = \text{None}$.
2. Repeat $V-1$ times: for every edge (u, v, w) , if $\text{dist}[u] + w < \text{dist}[v]$ then $\text{dist}[v] = \text{dist}[u] + w$ and $\text{prev}[v] = u$.
 - Why $V-1$ passes? The longest simple path uses at most $V-1$ edges; after $V-1$ relaxations, any further improvement indicates a negative cycle.
3. Do one more pass over edges. If any edge can still be relaxed, there is a **negative-weight cycle reachable from the source**. Mark nodes involved / affected.
4. (Optional) Propagate negative-cycle influence to all nodes reachable from those cycle nodes (their shortest path is undefined $\rightarrow -\infty$ conceptually).

Python implementation (robust, hashable node labels, negative-cycle reporting)

```
from typing import Iterable, Tuple, Any, Dict, List, Set, Optional

def bellman_ford(edges: Iterable[Tuple[Any, Any, float]], source: Any):
    """
    Bellman-Ford shortest paths.

    Args:
        edges: iterable of (u, v, weight) for directed edges. For
        undirected edges,
            include both (u,v,w) and (v,u,w).
        source: source node (hashable).

    Returns:
        dist: dict node -> shortest distance from source (float('inf') if
        unreachable)
        prev: dict node -> predecessor on shortest path (None if no
        predecessor)
        negative_cycle_nodes: set of nodes that are part of or reachable
        from a negative cycle
                                (empty set if no negative cycle reachable
        from source).

    Notes:
        - If negative_cycle_nodes is non-empty, shortest paths to those
        nodes are undefined.
    """
    edges_list = list(edges)

    # Collect nodes (ensure source exists)
    nodes = set()
    for u, v, _w in edges_list:
        nodes.add(u); nodes.add(v)
    nodes.add(source)

    dist: Dict[Any, float] = {n: float('inf') for n in nodes}
    prev: Dict[Any, Optional[Any]] = {n: None for n in nodes}
    dist[source] = 0.0

    V = len(nodes)

    # Relax edges up to V-1 times
    for i in range(V - 1):
```

```

        updated = False
        for u, v, w in edges_list:
            if dist[u] != float('inf') and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                prev[v] = u
                updated = True
        if not updated:
            # No change in this pass -> distances stabilized early
            break

    # Detect negative cycles: any edge that can still be relaxed indicates
    one
    neg_nodes: Set[Any] = set()
    for u, v, w in edges_list:
        if dist[u] != float('inf') and dist[u] + w < dist[v]:
            # v can be reduced further -> v (and u) are affected by a
    negative cycle
            neg_nodes.add(v)
            neg_nodes.add(u)

    # Propagate negativity: nodes reachable from neg_nodes are also
    affected
    if neg_nodes:
        # Repeat propagation up to V times (enough to reach all)
        for _ in range(V):
            for u, v, w in edges_list:
                if u in neg_nodes and v not in neg_nodes:
                    neg_nodes.add(v)

    return dist, prev, neg_nodes

def reconstruct_path(prev: Dict[Any, Optional[Any]], source: Any, target:
Any,
                    negative_cycle_nodes: Optional[Set[Any]] = None) ->
List[Any]:
    """
    Reconstruct path from source to target using prev dict.
    Returns empty list if target unreachable or if target is affected by a
    negative cycle.
    """
    if negative_cycle_nodes and target in negative_cycle_nodes:
        # Path undefined when target is affected by negative cycle
        return []

    path = []
    node = target
    while node is not None:
        path.append(node)
        if node == source:
            break
        node = prev.get(node)
    path.reverse()
    if not path or path[0] != source:
        return [] # unreachable
    return path

```

Detailed code explanation (line-by-line / block-by-block)

- **Collect nodes** — we build the `nodes` set from edges and ensure `source` is present. This lets the function work when nodes appear only as neighbors.
 - **Initialization:**
 - `dist[n] = +∞` for all nodes, `dist[source] = 0`.
 - `prev[n] = None` will hold the predecessor for path reconstruction.
 - **Relaxation loop (V-1 passes):**
 - For each pass, iterate every edge `(u, v, w)`. If `dist[u]` is finite and `dist[u] + w < dist[v]`, update `dist[v]` and `prev[v] = u`.
 - Use an updated flag to break early if no changes occur on a full pass (speeds up many graphs).
 - **Negative cycle detection:**
 - After V-1 passes, do one more pass through the edges. If any edge can still relax, a negative cycle is reachable from the source and influences some vertex.
 - We add both `u` and `v` to `neg_nodes` for safety (they are directly involved or next to the relaxing edge).
 - Then we **propagate**: repeatedly go over edges and add any vertex reachable from nodes in `neg_nodes`. After up to V iterations, we've added all nodes reachable from those cycles. These nodes' shortest distances are undefined (conceptually $-\infty$).
 - **Return:**
 - `dist` and `prev` give computed shortest distances for nodes **not** affected by negative cycles.
 - `negative_cycle_nodes` lists every node for which a finite shortest path is **not** defined because it can be improved indefinitely via cycles.
 - **Path reconstruction:**
 - If the `target` is unreachable (`dist` is $+\infty$) or is affected by a negative cycle, return `[]`.
 - Otherwise follow `prev` pointers back to the source and reverse.
-

Example 1 — graph without negative cycle

```
edges = [
    ('A', 'B', 6),
    ('A', 'D', 6),
    ('B', 'C', 5),
    ('C', 'D', 1),
    ('D', 'E', -2),
    ('B', 'E', -1)
]

dist, prev, neg = bellman_ford(edges, 'A')
print("Distances:", dist)
print("Negative cycle nodes:", neg)
print("Path A -> E:", reconstruct_path(prev, 'A', 'E', neg))
```

Expected outcome:

- Distances will be finite for reachable nodes (example: A:0, B:6, C:11, D:10, E:9 or similar depending on edge ordering).
- `neg` is empty (no negative cycle).
- Path A → E will be printed as a list like `['A', 'B', 'E']` if that is the shortest.

Example 2 — graph with a negative cycle

```
edges = [
    ('A', 'B', 1),
    ('B', 'C', 1),
    ('C', 'A', -3),    # cycle A->B->C->A has total weight -1
    ('C', 'D', 2)
]

dist, prev, neg = bellman_ford(edges, 'A')
print("Distances:", dist)
print("Negative cycle nodes:", neg)
print("Path A -> D:", reconstruct_path(prev, 'A', 'D', neg))
```

Expected outcome:

- `neg` will include A, B, C (and any nodes reachable from them).
- Distances for nodes in `neg` are unreliable (the algorithm may still show some numbers but they can be reduced without bound).
- `reconstruct_path` to D returns `[]` if D is affected by the negative cycle; otherwise if D is reachable only through the cycle it's also affected.

Use cases

- **Graphs with negative weights** (e.g., costs that can be negative).
- **Detecting negative cycles** — financial arbitrage detection (take `-log(rate)` and find negative cycles), protocol analysis.
- **Routing protocols** of distance-vector type (Bellman–Ford principles are used conceptually in e.g. RIP).
- **Preprocessing for Johnson's algorithm** (Bellman–Ford is used to compute reweighting potentials to remove negative weights, then run Dijkstra per node).

Tips, pitfalls & variants

- Bellman–Ford is **slower** ($O(VE)$) than Dijkstra, so prefer Dijkstra when weights are non-negative.
- Always **check for negative weights**: if none exist, use Dijkstra for much faster performance.

- When detecting a negative cycle, you often want **the actual cycle** — retrieving the cycle requires a little more bookkeeping (e.g., when an edge relaxes in the final pass, follow `prev` pointers V steps to find a node in the cycle and then output the cycle). The provided implementation focuses on identifying affected nodes.
 - For very large graphs where you need all-pairs shortest paths and negative edges exist, use **Johnson's algorithm** (Bellman–Ford + many Dijkstras) for better performance than repeated Bellman–Ford.
-

Floyd–Warshall Algorithm — Explanation + Python implementation

Short summary

Floyd–Warshall is an **all-pairs shortest paths** algorithm: it computes the shortest path distance between **every** pair of vertices in a weighted graph. It handles **negative edge weights** (unlike Dijkstra) and can detect negative cycles. The method uses dynamic programming with a triple loop over nodes and runs in $O(V^3)$ time.

Why it's used

- You need shortest distances **between every pair** of nodes (not just single-source).
 - You want a simple algorithm that handles **negative edge weights** (but not negative cycles).
 - It's very easy to implement and reason about (matrix-based DP).
 - Useful when V is small-to-moderate and you need complete distance information (dense graphs or small graphs).
-

When to use / limitations

- **Use Floyd–Warshall** when:
 - You require **all-pairs** shortest paths.
 - The graph is relatively small (because $O(V^3)$ grows quickly).
 - Edge weights may be negative but you still want to compute shortest paths (and detect negative cycles).
 - **Avoid** if:
 - Graph is large (V in thousands) — $O(V^3)$ becomes impractical.
 - Graph is sparse and you only need a few source nodes — use repeated Dijkstra (or Johnson's algorithm) instead.
-

Time & space complexity

- **Time:** $O(V^3)$ (three nested loops over vertices).
 - **Space:** $O(V^2)$ to store the distance matrix (and $O(V^2)$ extra if you store path reconstruction helpers like `next`).
 - Path reconstruction requires additional $O(V^2)$ space for the successor/predecessor matrix.
-

High-level idea

Let $\text{dist}[i][j]$ be the shortest distance from vertex i to vertex j using only intermediate vertices from the set $\{0 \dots k-1\}$. The classical DP recurrence:

$$\text{dist}_k[i][j] = \min(\text{dist}_{k-1}[i][j], \text{dist}_{k-1}[i][k] + \text{dist}_{k-1}[k][j])$$

We iterate k from 0 to $v-1$ and update dist in place. If at the end $\text{dist}[i][i] < 0$ for some i , a negative cycle exists reachable from i .

Python implementation (with path reconstruction & negative-cycle handling)

```
from typing import Iterable, Tuple, Any, List, Dict, Optional
import math

INF = float('inf')
NEG_INF = -float('inf')

def floyd_warshall(edges: Iterable[Tuple[Any, Any, float]]):
    """
    Floyd-Warshall algorithm for all-pairs shortest paths.

    Args:
        edges: iterable of (u, v, weight). For undirected graphs, include
        both directions.

    Returns:
        nodes: list of nodes (index -> node)
        dist: 2D list of distances (dist[i][j] is shortest distance from
        nodes[i] -> nodes[j])
            distances may be INF (no path) or NEG_INF (path can be made
            arbitrarily small due to a negative cycle).
        next_hop: 2D list used for path reconstruction. next_hop[i][j] is
        index of the next node
            after i on a shortest path to j, or None if no path or
            path affected by negative cycle.
        negative_cycle_nodes: set of node indices that are on or affected
        by negative cycles.
    """
    # 1) collect nodes and create index mapping
    nodes_set = set()
    edges_list = list(edges)
    for u, v, _w in edges_list:
        nodes_set.add(u); nodes_set.add(v)
    nodes = list(nodes_set)
    n = len(nodes)
    idx = {node: i for i, node in enumerate(nodes)}

    # 2) initialize distance and next matrices
    dist: List[List[float]] = [[INF]*n for _ in range(n)]
    next_hop: List[List[Optional[int]]] = [[None]*n for _ in range(n)]

    # distance to self is 0
```

```

for i in range(n):
    dist[i][i] = 0.0
    next_hop[i][i] = i

# fill direct edges (keep smallest weight if multiple edges)
for u, v, w in edges_list:
    i, j = idx[u], idx[v]
    if w < dist[i][j]:
        dist[i][j] = w
        next_hop[i][j] = j

# 3) core triple loop
for k in range(n):
    for i in range(n):
        if dist[i][k] == INF:
            continue
        for j in range(n):
            if dist[k][j] == INF:
                continue
            new_dist = dist[i][k] + dist[k][j]
            if new_dist < dist[i][j]:
                dist[i][j] = new_dist
                next_hop[i][j] = next_hop[i][k]

# 4) detect and propagate negative cycles
negative_cycle_nodes = set()
for v in range(n):
    if dist[v][v] < 0:
        negative_cycle_nodes.add(v)

# propagate -inf to any pair (i,j) that can go through a negative cycle
node
if negative_cycle_nodes:
    for k in negative_cycle_nodes:
        for i in range(n):
            if dist[i][k] == INF:
                continue
            for j in range(n):
                if dist[k][j] == INF:
                    continue
                dist[i][j] = NEG_INF
                next_hop[i][j] = None # no well-defined path

return nodes, dist, next_hop, negative_cycle_nodes

def reconstruct_path(nodes: List[Any], next_hop: List[List[Optional[int]]],
src: Any, dst: Any):
    """
    Reconstruct the path from src to dst using the next_hop matrix.
    Returns an empty list if no path or path undefined due to negative
    cycle.
    """
    idx = {node: i for i, node in enumerate(nodes)}
    if src not in idx or dst not in idx:
        return []

    i, j = idx[src], idx[dst]
    if next_hop[i][j] is None:
        return [] # no path or affected by negative cycle

```

```

path = [src]
while i != j:
    i = next_hop[i][j]
    if i is None:
        return [] # safety: undefined
    path.append(nodes[i])
    # guard against accidental infinite loops:
    if len(path) > len(nodes) + 5:
        return [] # something went wrong

return path

# Example usage
if __name__ == "__main__":
    edges = [
        ('A', 'B', 3),
        ('A', 'C', 8),
        ('B', 'C', 2),
        ('C', 'A', 4),
        ('B', 'D', 5),
        ('D', 'E', -6),
        ('E', 'B', 1) # creates a negative cycle: B->D->E->B has total
weight 0? (example)
    ]

    nodes, dist, next_hop, neg_nodes = floyd_warshall(edges)
    print("Nodes (index -> node):", nodes)
    print("Negative-cycle node indices:", neg_nodes)
    print("\nDistance matrix:")
    n = len(nodes)
    for i in range(n):
        row = []
        for j in range(n):
            d = dist[i][j]
            if d == INF:
                row.append("INF")
            elif d == NEG_INF:
                row.append("-INF")
            else:
                row.append(f"{d:.1f}")
        print(row)

    print("\nExample path A -> E:", reconstruct_path(nodes, next_hop, 'A',
'E'))

```

Detailed code explanation (block-by-block)

1) Node indexing

- Floyd–Warshall naturally works on indices $0 \dots n-1$. If input nodes are arbitrary hashable objects (strings, tuples), we map each node to an integer index (`idx`) and keep `nodes` list to convert back.

2) Matrices

- `dist[i][j]` holds the current best-known distance from `nodes[i]` to `nodes[j]`.
 - Initialize all to `INF`.
 - Set `dist[i][i] = 0.0`.
 - Fill in `dist[u][v] = weight` for each direct edge (keep smallest if multiple edges).
- `next_hop[i][j]` holds the **index** of the next vertex after `i` on a shortest path to `j`.
 - If there's a direct edge `i->j`, `next_hop[i][j] = j`.
 - If no path, `next_hop[i][j] = None`.
 - When updating via an intermediate `k`, we set `next_hop[i][j] = next_hop[i][k]`.

3) Triple loop

- For each `k` (allowed intermediate nodes `0..k`), for each `(i, j)` we check whether going `i -> k -> j` improves the `i -> j` distance:
 - if `dist[i][k] + dist[k][j] < dist[i][j]` then update `dist[i][j]` and set `next_hop[i][j] = next_hop[i][k]`.
- We skip updates when either `dist[i][k]` or `dist[k][j]` is `INF` to avoid overflow/incorrect arithmetic.

4) Negative-cycle detection

- If after the algorithm `dist[v][v] < 0` for some `v`, a negative cycle exists and is reachable from `v`.
- Pairs `(i, j)` that can pass through such a node `k` have undefined finite shortest distance (you can keep reducing cost by looping the negative cycle). We mark these distances as `-INF` (or handle differently depending on application) and set `next_hop[i][j] = None` to indicate no well-defined finite path.

5) Path reconstruction

- Use `next_hop` to walk from `src` to `dst`:
 - Start `i = idx[src]`. While `i != idx[dst]`: set `i = next_hop[i][dst]` and append `nodes[i]`.
 - If `next_hop[i][dst]` is `None` we return empty list (no path or negative-cycle affected).
- This reconstruction is $O(\text{path_length})$.

Use cases

- **All-pairs shortest paths** in small to medium graphs.
- **Graphs with negative edge weights** (but no negative cycles), e.g., certain economic models.
- Computing **reachability** / **transitive closure** (if you set weight 1 for edges and 0 on diagonal, iterating min-plus can compute reachability; Warshall algorithm is an adaptation).

- Useful in dynamic programming solutions where you need distances between all pairs for later composition (e.g., some routing, clustering, or network analysis tasks).
 - **Pedagogical / prototyping:** easy to code and reason about.
-

Practical tips & variants

- For **sparse graphs** and large V , prefer Johnson's algorithm (Bellman–Ford once + V Dijkstra runs) which is usually faster in practice.
 - For **only single-source** queries, use Dijkstra (if weights non-negative) or Bellman–Ford (if negatives).
 - When implementing, watch for floating-point issues if weights are floats — comparisons with `INF` must be explicit.
 - If you only need distances (not reconstructable paths), you can skip the `next_hop` matrix to save memory — still $O(V^2)$ space for distances.
 - You can adapt Floyd–Warshall to compute other path properties (e.g., count of shortest paths, lexicographically smallest path) with extra bookkeeping.
-

Topological Sort — Explanation + Python implementations

Short summary

Topological sort orders the vertices of a **directed acyclic graph (DAG)** so that for every directed edge $u \rightarrow v$, u comes before v in the ordering. It's used whenever you must linearize tasks with dependencies (build systems, course prerequisites, job scheduling, etc.). Topological order exists **iff** the graph is a DAG.

Two common algorithms

1. **Kahn's algorithm** (BFS / indegree) — iterative, easy to reason about, detects cycles by checking whether all nodes are output.
2. **DFS-based** (reverse postorder) — run DFS, push nodes after visiting all descendants, then reverse the result. Use a recursion stack or color-marking to detect cycles.

Both run in $O(V + E)$ time and use $O(V + E)$ space (adjacency list + helper arrays).

Kahn's algorithm (indegree / queue)

Why it works

Start from nodes with no incoming edges (indegree 0). Removing them removes outgoing edges and may create new indegree-0 nodes. Repeating this produces a valid order. If you cannot visit all nodes, a cycle exists.

Python code (Kahn)

```
from collections import deque, defaultdict
from typing import Dict, List, Any, Tuple

def topo_kahn(edges: List[Tuple[Any, Any]]) -> List[Any]:
    """
    edges: list of (u, v) directed edges
    Returns topological order list (empty list if cycle detected).
    """
    # Build adjacency list and indegree map
    adj: Dict[Any, List[Any]] = defaultdict(list)
    indegree: Dict[Any, int] = defaultdict(int)
    nodes = set()
    for u, v in edges:
        nodes.add(u); nodes.add(v)
        adj[u].append(v)
        indegree[v] += 1
        indegree.setdefault(u, indegree.get(u, 0)) # ensure u present

    # Initialize queue with all indegree-0 nodes
    q = deque([n for n in nodes if indegree.get(n, 0) == 0])
```

```

order: List[Any] = []

while q:
    u = q.popleft()
    order.append(u)
    for v in adj[u]:
        indegree[v] -= 1
        if indegree[v] == 0:
            q.append(v)

# If order doesn't contain all nodes, there's a cycle
if len(order) != len(nodes):
    return [] # cycle detected
return order

# Example:
# edges = [('A', 'C'), ('B', 'C'), ('C', 'D'), ('D', 'E')]
# topo_kahn(edges) -> e.g. ['A', 'B', 'C', 'D', 'E'] (A and B order may swap)

```

Step-by-step trace (Kahn) on example A->C, B->C, C->D:

- indegree: A:0, B:0, C:2, D:1
- queue init: [A, B]
- pop A → order [A], decrease indegree(C)=1
- pop B → order [A,B], decrease indegree(C)=0 → push C
- pop C → order [A,B,C], decrease indegree(D)=0 → push D
- pop D → order [A,B,C,D] done.

DFS-based algorithm (reverse postorder)

Why it works

A DFS finishes (postorder) a node only after all nodes reachable from it are finished. If you append nodes to a list when finishing them and then reverse that list, you get a topological order.

Cycle detection

Use a `visiting` set (recursion stack) or 3-color scheme

(0=unvisited, 1=visiting, 2=visited). If you see a `visiting` node again, there's a back-edge → cycle.

Python code (DFS)

```

from collections import defaultdict
from typing import Dict, List, Any, Tuple

def topo_dfs(edges: List[Tuple[Any, Any]]) -> List[Any]:
    """
    Returns topological order list, or [] if a cycle is detected.
    """
    adj: Dict[Any, List[Any]] = defaultdict(list)
    nodes = set()
    for u, v in edges:
        nodes.add(u); nodes.add(v)

```

```

        adj[u].append(v)

    visited: Dict[Any, int] = {n: 0 for n in nodes} #
    0=unvisited,1=visiting,2=visited
    order: List[Any] = []
    cycle_found = False

    def dfs(u: Any):
        nonlocal cycle_found
        if cycle_found:
            return
        visited[u] = 1 # visiting
        for v in adj[u]:
            if visited[v] == 0:
                dfs(v)
                if cycle_found:
                    return
            elif visited[v] == 1:
                # back-edge found -> cycle
                cycle_found = True
                return
        visited[u] = 2 # finished
        order.append(u)

    for n in nodes:
        if visited[n] == 0:
            dfs(n)
            if cycle_found:
                return []

    order.reverse()
    return order

# Example:
# edges = [('A','C'), ('B','C'), ('C','D')]
# topo_dfs(edges) -> ['A','B','C','D'] or ['B','A','C','D']

```

Trace (DFS) on A->C, B->C, C->D:

- Start A: dfs(A) → visit C → visit D → finish D (append), finish C (append), finish A (append) => partial order [D,C,A]
- Start B: dfs(B) → reach C (already finished), finish B (append) => [D,C,A,B]
- Reverse => [B,A,C,D] (A and B may swap; both valid)

Detecting cycles (explicit)

- **Kahn:** after algorithm, if produced order length < |V|, a cycle exists.
- **DFS:** detect back-edge during traversal (visited==1).

I gave both implementations returning [] when a cycle is present. If you want the actual cycle nodes/edges, I can add code to extract the cycle.

Complexity

- **Time:** $O(V + E)$ for both Kahn and DFS.
 - Building adjacency lists and indegree arrays: $O(E + V)$.
 - Each edge is considered exactly once when decreasing indegree (Kahn) or when exploring neighbors (DFS).
 - **Space:** $O(V + E)$ for adjacency + bookkeeping (indegree/visited/stack/order).
-

When to use which algorithm

- **Kahn's:**
 - Good when you prefer an iterative BFS-like solution.
 - Easy to stream or implement non-recursively.
 - Useful when you want to produce *any* topological order and also want trivial cycle detection.
 - **DFS-based:**
 - Natural if you already use DFS for other graph analyses.
 - Slightly simpler to get reverse-postorder.
 - Beware recursion depth on very deep graphs (use iterative DFS or increase recursion limit for Python).
-

Use cases

- **Build systems / compilation order** (source files with dependencies).
 - **Task scheduling** (jobs with prerequisites).
 - **Course prerequisite resolution** (a valid class order).
 - **Package / dependency managers** (install order).
 - **Data pipeline orchestration** (DAGs in workflow engines like Airflow).
 - **Topological sorting of partial orders** in combinatorics, scheduling, and serialization.
-

Practical notes & tips

- Graph must be **directed**; topological order doesn't apply to undirected graphs (except trivial cases).
- If multiple valid orders exist, algorithms return one — Kahn's result depends on queue order; DFS depends on node iteration order. If you need a *lexicographically smallest* topo order, push indegree-0 nodes into a min-heap (priority queue) instead of a plain queue.
- For very large graphs, prefer adjacency lists; avoid adjacency matrices.
- If you need to enumerate **all** topological sorts, that's an exponential problem (use backtracking with indegree updates) — feasible only for very small graphs.

All-in-one example (Kahn + detect cycle + lexicographic variant)

```
import heapq
from collections import defaultdict
from typing import List, Tuple, Any

def topo_kahn_any(edges: List[Tuple[Any, Any]]):
    # standard Kahn
    return topo_kahn(edges)

def topo_kahn_lexicographic(edges: List[Tuple[str, str]]):
    # returns lexicographically smallest topological order (strings or
    comparable)
    adj = defaultdict(list)
    indeg = defaultdict(int)
    nodes = set()
    for u, v in edges:
        nodes.add(u); nodes.add(v)
        adj[u].append(v)
        indeg[v] += 1
        indeg.setdefault(u, indeg.get(u, 0))

    heap = [n for n in nodes if indeg[n] == 0]
    heapq.heapify(heap)
    order = []
    while heap:
        u = heapq.heappop(heap)
        order.append(u)
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                heapq.heappush(heap, v)
    return order if len(order) == len(nodes) else [] # empty => cycle
```

Flood Fill — explanation + Python implementations

Short summary

Flood Fill fills a **connected region** of a grid/image starting from a seed pixel with a new color/value. It replaces every cell reachable from the seed by moving along allowed neighbors (usually 4-directional: up, down, left, right — sometimes 8-directional including diagonals) that have the same original color.

Think of the “paint bucket” tool in graphics editors.

Why it’s used

- To recolor a connected region of pixels (paint-bucket).
 - To **label connected components** in binary/segmented images (preprocessing for computer vision).
 - To detect area/region size and boundaries (game maps, flood-fill area checks).
 - As a building block in image processing, pathfinding, terrain generation, and graph connectivity tasks.
-

When to use (vs alternatives)

- Use flood fill when you need to operate on all cells in a connected region reachable by equality of value/color.
 - For **small-to-moderate** regions and grids, simple DFS/BFS implementations are fine.
 - For very large images or performance-critical applications, use optimized variants (scanline flood fill, union-find labeling, or specialized image libraries that operate in C).
 - If you need **connected-component labeling** over the whole image, use multi-pass algorithms or union-find approaches instead of repeated flood fill calls.
-

Time & space complexity

Let $R = \text{rows}$, $C = \text{cols}$, and $N = R * C$.

- **Time:** $O(K)$ where K is the number of cells visited (cells in the filled region). Worst-case $K = N$, so $O(N)$.
- **Space:** $O(K)$ for recursion stack or queue/stack used to track frontier (worst-case $O(N)$).

- Recursive implementations risk hitting recursion depth limits for large κ ; iterative (stack/queue) avoids that.
-

Variants

- **Recursive DFS** — very simple, but may overflow recursion stack on big regions.
 - **Iterative BFS (queue)** — breadth-first filling; safe for deep regions.
 - **Iterative DFS (stack)** — similar to recursive DFS but avoids recursion limits.
 - **Scanline flood fill** — faster in practice for large contiguous horizontal spans (fills whole scanlines); more code but fewer pushes/pops.
 - **Union-Find** — useful when labeling all components in the entire image (connected-component labeling).
 - **Image-library methods** — Pillow/OpenCV have optimized routines and are recommended for production.
-

Correctness details / pitfalls

- **Early exit:** if `newColor == originalColor`, do nothing — otherwise you will loop infinitely or keep processing.
 - **Bounds checks:** always ensure neighbor coordinates are inside the grid/image.
 - **Marking visited:** change color to `newColor` as soon as you visit/enqueue a cell to avoid revisiting duplicates.
 - **Connectivity:** 4-direction vs 8-direction determines which neighbors count as connected. Choose per problem.
-

Implementations (clear, explained)

1) Simple recursive DFS (easy to read — watch recursion depth)

```
def flood_fill_recursive(grid, sr, sc, new_color):
    """
    grid: list of lists, grid[r][c] is a color/value
    sr, sc: start row, col
    new_color: value to fill

    Modifies grid in-place and returns it.
    """
    rows = len(grid)
    cols = len(grid[0]) if rows else 0
    orig = grid[sr][sc]

    # Early exit: nothing to do
    if orig == new_color:
        return grid
```

```

def dfs(r, c):
    # Bounds check
    if r < 0 or r >= rows or c < 0 or c >= cols:
        return
    # Only fill cells that have the original color
    if grid[r][c] != orig:
        return

    # Fill current cell
    grid[r][c] = new_color

    # Explore 4 neighbors (up, down, left, right)
    dfs(r + 1, c)
    dfs(r - 1, c)
    dfs(r, c + 1)
    dfs(r, c - 1)

dfs(sr, sc)
return grid

```

Line-by-line notes

- `orig = grid[sr][sc]` stores the color to replace.
- `if orig == new_color:` avoids infinite work when nothing should change.
- `dfs` checks bounds and whether the current cell still has the original color. If so, it sets it to `new_color` and recurses to neighbors.
- This visits each cell in the connected region exactly once (if no cycles of color change), but recursion depth equals the longest path in region — might overflow Python recursion for large regions.

2) Iterative BFS (safer for deep/large regions; preferable)

```

from collections import deque

def flood_fill_bfs(grid, sr, sc, new_color):
    rows = len(grid)
    cols = len(grid[0]) if rows else 0
    orig = grid[sr][sc]

    if orig == new_color:
        return grid

    q = deque()
    q.append((sr, sc))
    # Mark visited by setting to new_color immediately to avoid re-adding
    grid[sr][sc] = new_color

    while q:
        r, c = q.popleft()
        for dr, dc in ((1,0), (-1,0), (0,1), (0,-1)): # 4-connected
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == orig:
                grid[nr][nc] = new_color # mark visited
                q.append((nr, nc))
    return grid

```

Why mark on enqueue?

Setting `grid[nr][nc] = new_color` when we enqueue prevents pushing the same cell multiple times (saves memory/time). You still visit each cell exactly once.

3) Iterative DFS (stack) — similar to BFS, sometimes faster

```
def flood_fill_stack(grid, sr, sc, new_color):
    rows = len(grid)
    cols = len(grid[0]) if rows else 0
    orig = grid[sr][sc]
    if orig == new_color:
        return grid

    stack = [(sr, sc)]
    grid[sr][sc] = new_color

    while stack:
        r, c = stack.pop()
        for dr, dc in ((1,0), (-1,0), (0,1), (0,-1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == orig:
                grid[nr][nc] = new_color
                stack.append((nr, nc))
    return grid
```

4) Flood fill on a Pillow image (practical image example)

```
# Requires: pip install pillow
from PIL import Image

def flood_fill_pil(img: Image.Image, x: int, y: int, new_color):
    """
    img: PIL Image (mode 'RGB' or 'RGBA' or 'L', etc.)
    x, y: pixel coordinates (x horizontal, y vertical)
    new_color: tuple or int matching image mode, e.g. (255,0,0) for RGB
    Returns modified image (in-place).
    """
    pixels = img.load()
    w, h = img.size
    orig = pixels[x, y]

    if orig == new_color:
        return img

    stack = [(x, y)]
    pixels[x, y] = new_color

    while stack:
        cx, cy = stack.pop()
        for nx, ny in ((cx+1, cy), (cx-1, cy), (cx, cy+1), (cx, cy-1)):
            if 0 <= nx < w and 0 <= ny < h and pixels[nx, ny] == orig:
                pixels[nx, ny] = new_color
                stack.append((nx, ny))
    return img
```

Notes

- Pixel values may be tuples (R, G, B) or integers (grayscale). Use a `new_color` that matches the image mode.
 - Converting image to `RGBA` or `RGB` first can simplify handling.
-

Example (grid) — quick demo

Input grid:

```
grid = [  
    [1,1,1],  
    [1,1,0],  
    [1,0,1]  
]  
flood_fill_bfs(grid, 1, 1, 2)  
# result:  
# [  
#   [2,2,2],  
#   [2,2,0],  
#   [2,0,1]  
# ]
```

Seed $(1, 1)$ had value 1, so all connected 1's reachable via 4-neighbor connectivity become 2.

Practical tips & optimizations

- **Always early-exit** if `orig == new_color`. This avoids altering the grid and avoids infinite loops.
 - **Prefer iterative** implementations for production to avoid recursion-depth issues.
 - **Marking on enqueue** (BFS) or on push (DFS stack) prevents duplicate pushes and reduces memory.
 - For **large contiguous regions**, implement **scanline flood fill** (fills horizontal segments and then checks above/below spans) — fewer push/pop operations and much faster in practice for large blocks.
 - For **color images** compare pixels exactly (tuples). If using floating colors, be careful with equality; consider thresholding.
 - For repeated labeling across the whole image, use a **connected-component labeling** algorithm (two-pass union-find) rather than many flood fills.
-

Use cases (concise)

- Paint/bucket fill in image editors.

- Connected-component labeling for segmentation.
 - Region counting and area measurement (e.g., count lakes/patches in maps).
 - Game logic: reachable area, flood-fill puzzles, territory coloring.
 - Preprocessing for computer vision tasks (remove small islands, fill holes).
 - Simple mask-growing segmentation.
-

When flood fill is NOT ideal

- If you need to fill many independent regions across a huge image, a full connected-component labeling algorithm is more efficient.
- When you need very fast low-level operations on large images — prefer OpenCV or Pillow routines implemented in C.
- If you need to fill based on color similarity (not exact equality), use region-growing with thresholds or morphological methods instead.

Lee's algorithm — explanation + Python implementation

Lee's algorithm is **BFS on a grid** used to find the **shortest path (in number of steps)** from a start cell to a goal cell in an unweighted grid/maze where some cells are blocked. It guarantees the shortest path (by steps) because BFS explores cells in increasing distance order.

Why it's used

- It finds the **shortest path** on a discrete grid/maze (when all moves cost the same).
 - It's conceptually simple and easy to implement with a queue (`deque`).
 - It works for pathfinding in mazes, routing on grids, board games, robotics discretized maps, and many grid-based problems in programming contests.
-

When to use / Limitations

- **Use Lee** when the grid is unweighted (every move has equal cost) and you need the shortest number-of-steps path.
 - **Do not use** Lee when edges have different weights — then use Dijkstra (or 0-1 BFS for weights 0/1).
 - For huge grids with memory constraints, you may need memory-optimized variants or external storage.
-

Time & Space Complexity

Let R = number of rows, C = number of columns, $N = R * C$.

- **Time:** $O(N)$ worst-case — each cell is enqueued/visited at most once and each neighbor check is constant-time.
 - **Space:** $O(N)$ for `dist/visited/parent` arrays and for the queue in the worst case.
-

High-level idea

1. Use a queue (BFS) starting from the seed cell.
2. Mark visited and record parent/distance for each neighbor you push into the queue.

3. When you first reach the goal, you have the shortest path — reconstruct it using parent pointers.
 4. If the queue empties without reaching the goal, there is no path.
-

Python implementation (4-directional with optional 8-direction mode)

```
from collections import deque
from typing import List, Tuple, Optional

Coord = Tuple[int, int]

def lee_shortest_path(
    grid: List[List[int]],
    start: Coord,
    goal: Coord,
    allow_diagonal: bool = False
) -> Tuple[List[Coord], int]:
    """
    Find shortest path (list of (r,c) coords) from start to goal on a grid
    using Lee's algorithm (BFS).
    grid: 2D list where 0 = free cell, 1 = blocked cell (change as needed).
    start, goal: (row, col) 0-based coordinates.
    allow_diagonal: if True, allows 8-directional moves (including
    diagonals).
    Returns: (path, steps) where path is list of coords from start to goal
    inclusive,
            and steps is number of moves (len(path)-1). If unreachable,
    returns ([], -1).
    """
    rows = len(grid)
    cols = len(grid[0]) if rows else 0
    sr, sc = start
    gr, gc = goal

    # Basic validation
    if not (0 <= sr < rows and 0 <= sc < cols):
        raise ValueError("start out of bounds")
    if not (0 <= gr < rows and 0 <= gc < cols):
        raise ValueError("goal out of bounds")
    if grid[sr][sc] != 0 or grid[gr][gc] != 0:
        # blocked start or goal
        return [], -1

    if start == goal:
        return [start], 0

    # Directions: 4-neighborhood by default; add diagonals if requested
    dirs = [(1,0), (-1,0), (0,1), (0,-1)]
    if allow_diagonal:
        dirs += [(1,1), (1,-1), (-1,1), (-1,-1)]

    # Distance array initialized to -1 (unvisited)
    dist = [[-1] * cols for _ in range(rows)]
```

```

    parent: List[List[Optional[Coord]]] = [[None] * cols for _ in
range(rows)]

    q = deque()
    q.append((sr, sc))
    dist[sr][sc] = 0
    parent[sr][sc] = None # start has no parent

    while q:
        r, c = q.popleft()
        # Explore neighbors
        for dr, dc in dirs:
            nr, nc = r + dr, c + dc
            # bounds check
            if not (0 <= nr < rows and 0 <= nc < cols):
                continue
            # only step into free and unvisited cells
            if grid[nr][nc] == 0 and dist[nr][nc] == -1:
                dist[nr][nc] = dist[r][c] + 1
                parent[nr][nc] = (r, c)
                # If goal found we can reconstruct path early (optional
speed-up)
                if (nr, nc) == (gr, gc):
                    return _reconstruct_path(parent, start, goal),
dist[nr][nc]
                q.append((nr, nc))

    # goal not reached
    return [], -1

def _reconstruct_path(parent: List[List[Optional[Coord]]],
                      start: Coord, goal: Coord) -> List[Coord]:
    """Reconstruct path from start to goal using parent pointers (returns
list of coords)."""
    path: List[Coord] = []
    cur = goal
    while cur is not None:
        path.append(cur)
        if cur == start:
            break
        cur = parent[cur[0]][cur[1]]
    path.reverse()
    # Verify that path starts with start (otherwise goal unreachable)
    if not path or path[0] != start:
        return []
    return path

# Example usage
if __name__ == "__main__":
    grid = [
        [0,0,0,0,1],
        [1,1,0,1,0],
        [0,0,0,0,0],
        [0,1,1,1,0],
        [0,0,0,0,0],
    ]
    start = (0, 0)
    goal = (4, 4)

```

```
path, steps = lee_shortest_path(grid, start, goal,
allow_diagonal=False)
print("Path:", path)
print("Steps:", steps)
```

Detailed code explanation (block-by-block)

Validation & early returns

- We check bounds and whether start/goal are blocked. If start==goal we return immediately with a 0-step path.

Directions

- `dirs = [(1,0), (-1,0), (0,1), (0,-1)]` covers 4-neighborhood. If `allow_diagonal=True` we append the 4 diagonal directions.
- *Note:* when allowing diagonal moves, consider "corner cutting" — moving diagonally between two orthogonally blocked cells might be disallowed in some problems. You can add a check to prevent diagonal moves that pass through corners blocked on both orthogonal neighbors.

dist and parent

- `dist[r][c] == -1` means unvisited. When we visit a neighbor we set `dist[nr][nc] = dist[r][c] + 1`.
- `parent[nr][nc] = (r, c)` lets us reconstruct the path backward from the goal.

BFS loop

- `while q: r,c = q.popleft()`
- For each neighbor: check bounds and `grid[nr][nc] == 0` (free) and not yet visited.
- We mark `dist` & `parent` **when we enqueue** the neighbor (not when dequeuing). This avoids multiple enqueues of the same cell and ensures $O(N)$ time.
- If the neighbor equals the goal, we reconstruct and return early. Otherwise we append the neighbor to the queue.

Reconstruction

- `_reconstruct_path` follows parent pointers from goal back to start, builds the reversed path, then reverses it. If path doesn't start at start, the goal was unreachable — return `[]`.

Example (walkthrough)

Using the example grid in code:

- BFS will explore cells layer by layer (distance 0, then 1, then 2...).
 - The first time we pop the goal (or discover it), the `dist` value is the shortest number of moves.
 - `_reconstruct_path` will return the actual coordinates visited in the shortest path.
-

Variants & useful adaptations

- **Multi-source BFS:** push multiple start coordinates into the initial queue with distance 0 to compute shortest distance from the nearest source (useful for fire spread, multi-start routing).
 - **Distance map:** run BFS from a source (or multi-source) and return the whole `dist` matrix — gives shortest distance to every reachable cell.
 - **Avoid corner-cutting** for diagonal moves: when moving diagonally (`dr, dc`) ensure at least one of the orthogonal neighbors is free (or ensure both are free) depending on problem rules.
 - **Weighted grid:** if moves have different costs, use Dijkstra (priority queue) or 0-1 BFS for costs 0/1. Lee's BFS is only for uniform-cost moves.
 - **Memory optimization:** if you only need distances and not parents, skip parent array to halve memory. If you only need reachability, use boolean `visited` instead of `dist`.
-

Practical tips & pitfalls

- **Mark visited on enqueue** (not on dequeue) — prevents pushing the same cell many times.
 - **Check start/goal are free** before starting.
 - For **very large grids**, ensure you don't keep giant parent arrays if memory is constrained; consider reconstructing path differently or compressing parents.
 - **Corner-cutting rules** matter for diagonal movement — clarify problem spec.
 - **Use iterative BFS** rather than recursion to avoid recursion depth issues.
-

Use cases

- Maze shortest path (classic maze solver).
- Grid-based path planning in robotics on occupancy grids (discrete maps).
- Game AI for grid movement (when cost is uniform).
- Nearest-signal/nearest-source calculations (multi-source BFS).
- Flood-fill-like distance propagation (distance transform with Manhattan metric).